

## Improved Accuracy

```
===== IMPROVED MODEL RESULTS =====
```

|              |          |   |        |              |
|--------------|----------|---|--------|--------------|
| <u>Train</u> | accuracy | : | 0.9280 | (was 0.7341) |
| Dev          | accuracy | : | 0.6337 | (was 0.6931) |

  

| Classification | Report:   |        |          |         |  |
|----------------|-----------|--------|----------|---------|--|
|                | precision | recall | f1-score | support |  |
| G              | 0.91      | 0.77   | 0.83     | 26      |  |
| F              | 0.89      | 1.00   | 0.94     | 55      |  |
| E              | 0.65      | 0.73   | 0.69     | 41      |  |
| D              | 0.20      | 0.12   | 0.15     | 16      |  |
| C              | 0.27      | 0.21   | 0.24     | 14      |  |
| B              | 0.42      | 0.65   | 0.51     | 20      |  |
| A              | 0.38      | 0.18   | 0.24     | 17      |  |
| S              | 0.17      | 0.15   | 0.16     | 13      |  |
| accuracy       |           |        | 0.63     | 202     |  |
| macro avg      | 0.49      | 0.48   | 0.47     | 202     |  |
| weighted avg   | 0.61      | 0.63   | 0.61     | 202     |  |

The improved model achieved a train accuracy of 0.9280 (up from 0.7341), but dev accuracy dropped slightly from 0.6931 to 0.6337, which indicates overfitting — the model memorized training data but generalizes worse on unseen data.

### Key Problems

1. D, C, A, S tiers are failing with F1-scores below 0.25. These are mid-to-high tiers with only 13–20 samples each — the model has too little data to learn them properly.
2. Class imbalance is severe: F tier has 55 samples while S tier has only 13. The model is biased toward predicting common tiers.
3. Overfitting: the 29% gap between train and dev accuracy means the model is too complex for the dataset size.

## AI Suggestion

```
To address the issues and improve the model's performance:

1. Address class imbalance:
    * Use techniques like oversampling the minority classes, undersampling the majority classes, or generating synthetic samples using SMOTE or similar algorithms.
    * Consider using class weights during training to give more importance to the minority classes.
2. Feature engineering:
    * Explore additional features that can help distinguish between the different tiers, such as Pokémon stats, move sets, or evolutionary lines.
    * Consider using feature selection techniques to identify the most relevant features for the classification task.
3. Model selection and hyperparameter tuning:
    * Try different models, such as Random Forest, Gradient Boosting, or Support Vector Machines, which might be better suited for the classification task.
    * Perform hyperparameter tuning using techniques like grid search, random search, or Bayesian optimization to find the optimal combination of hyperparameters for the chosen model.
4. Regularization and early stopping:
    * Use regularization techniques, such as L1 or L2 regularization, to prevent overfitting.
    * Implement early stopping to prevent overfitting by stopping the training process when the model's performance on the dev set starts to degrade.
5. Collect more data:
    * If possible, collect more data to increase the size of the minority classes and reduce the class imbalance.
6. Ensemble methods:
    * Consider using ensemble methods, such as bagging or boosting, to combine the predictions of multiple models and improve overall performance.

Example code for some of these suggestions:
...python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from imblearn.over_sampling import SMOTE

# Oversample the minority classes using SMOTE
smote = SMOTE(random_state=42)
X_res, y_res = smote.fit_resample(X_train, y_train)

# Define a Random Forest Classifier
rfc = RandomForestClassifier(n_estimators=100, random_state=42)

Sending data to Groq for analysis... Please wait.

===== GROQ SUGGESTIONS =====

Analysis of the Classification Report

Based on the provided classification report, we can identify the tiers that are performing poorly:

1. Tier D: precision (0.25), recall (0.19), f1-score (0.21) - The model is struggling to correctly classify Tier D Pokémon, with a low precision and recall.
2. Tier C: precision (0.29), recall (0.14), f1-score (0.19) - Similar to Tier D, the model is having trouble accurately classifying Tier C Pokémon.
3. Tier A: precision (0.38), recall (0.20), f1-score (0.33) - Tier A Pokémon are also being misclassified, with a relatively low precision and recall.
4. Tier S: precision (0.60), recall (0.23), f1-score (0.33) - While the precision is moderate, the recall is very low, indicating that the model is missing many Tier S Pokémon.

Causes of Poor Performance

Several factors might be contributing to the model's struggle:

1. Class imbalance: The dataset appears to be imbalanced, with some tiers (e.g., F) having a much larger number of samples than others (e.g., D, C, S). This can lead to the model being biased towards the majority classes and performing poorly on the minority classes.
2. Lack of features or information: The model might not have access to relevant features or information that can help distinguish between the different tiers. This could be due to the quality of the data or the features extracted from it.
3. Model capacity: The MLPClassifier might not have sufficient capacity to learn complex relationships between the features and the target variable. This could be due to the number of hidden layers, the number of neurons in each layer, or the choice of activation functions.
4. Overfitting or underfitting: The model might be overfitting or underfitting the training data, leading to poor performance on the dev set.

Actionable Suggestions to Improve the Model
```

## Root Causes Identified by Groq

1. Class imbalance — Tier F dominates the dataset while D, C, A, S are severely underrepresented, causing the model to favor majority classes
2. Lack of relevant features — Current stats may not fully capture what separates mid-to-high tiers from each other
3. Insufficient model capacity — A single hidden layer of 10 neurons is too simple for an 8-class problem with complex boundaries
4. Overfitting — The large gap between train (0.93) and dev (0.63) accuracy confirms the model is not generalizing well

## Groq's Recommended Solutions

1. Fix class imbalance using SMOTE to synthetically oversample minority tiers, or apply `class_weight='balanced'`
2. Add more features such as move sets or evolutionary lines to better distinguish tiers
3. Try stronger models like Random Forest or Gradient Boosting which handle imbalanced data better than MLP
4. Add regularization via L1/L2 and implement early stopping to reduce overfitting
5. Use hyperparameter tuning with GridSearchCV or Bayesian optimization
6. Use ensemble methods like bagging or boosting to combine multiple models

## Conclusion

The core problem is not the model architecture itself but the data quality and class imbalance. Applying SMOTE and switching to a Random Forest or Gradient Boosting classifier would likely yield the biggest improvement in dev accuracy for the underperforming tiers.